# Comparative Sentiment Analysis of YouTube Transcripts and User Comments: Failure Modes and Interpretability in Public Discourse

Gabu Sai Yamini Devi

Independent Researcher

saiyaminireddy@gmail.com

February 2026

**Abstract**

Sentiment analysis of user-generated content on video-sharing platforms presents a unique methodological challenge: the sentiment expressed in a video's spoken content (transcript) does not necessarily align with the sentiment expressed by its audience in the comments section. This paper presents a dual-model sentiment analysis system combining LSTM neural networks for comment classification with VADER lexicon-based analysis for transcript sentiment. Through deployment of a full-stack web application (Python/Flask backend, MySQL storage, interactive visualization), we demonstrate the feasibility of comparative transcript-comment sentiment analysis. A pilot study analyzing 12 YouTube videos across five content domains reveals divergence patterns: product reviews show transcripts more positive than comments (reflecting promotional content versus user experience), political commentary shows the opposite (audience supporting critical creator stance), while educational content shows minimal divergence. We document domain-specific failure modes including ASR transcription noise affecting VADER analysis and sarcasm degrading LSTM accuracy. The system architecture, dual-model rationale, and pilot findings provide a foundation for larger-scale analysis of creator-audience sentiment relationships. We release our implementation at https://github.com/yamireddy04/Youtube_Sentiment_analysis_ai.

**Keywords:** sentiment analysis, YouTube, LSTM, VADER, transcript analysis, comment analysis, neural networks, lexicon-based methods, failure modes, interpretability, public discourse

## Notation and Symbols

| Symbol | Definition |
|---|---|
| $\mathcal{V}$ | Set of YouTube videos |
| $v_i$ | Individual video $i \in \mathcal{V}$ |
| $T(v_i)$ | Transcript of video $v_i$ |
| $C(v_i)$ | Set of comments for video $v_i$ |
| $c_j$ | Individual comment $j \in C(v_i)$ |
| $S_{\text{LSTM}}(\cdot)$ | LSTM sentiment classifier for comments |
| $S_{\text{VADER}}(\cdot)$ | VADER sentiment analyzer for transcripts |
| $Y_s$ | Sentiment label space $\{\text{negative}, \text{neutral}, \text{positive}\}$ |
| $\delta(v_i)$ | Transcript-comment sentiment divergence for video $v_i$ |
| $\bar{\delta}$ | Mean divergence across a set of videos |
| $P_T, N_T, U_T$ | Positive, negative, neutral percentages for transcripts |
| $P_C, N_C, U_C$ | Positive, negative, neutral percentages for comments |
| $K$ | Number of comments fetched per video |
| $L$ | Maximum sequence length for LSTM input |
| $V$ | Vocabulary size for LSTM tokenizer |

Table 1: Notation used throughout the paper.

# 1 Introduction

YouTube, as the world's largest video-sharing platform, generates a unique dual-signal corpus: creator-produced spoken content captured in video transcripts, and audience-produced reactions expressed in the comments section. While sentiment analysis of user-generated text has been extensively studied [8, 5], the comparative analysis of creator sentiment versus audience sentiment within the same content context remains underexplored.

This divergence has practical significance. A political commentary video may express measured analysis while generating polarized audience comments. An educational tutorial may maintain neutral instructional tone while generating frustrated learner feedback. A product review may express enthusiasm while comments reveal widespread dissatisfaction. Treating transcript and comment sentiment as equivalent signals fundamentally mischaracterizes the affective landscape of online video discourse.

The challenge is compounded by the distinct linguistic characteristics of each text type. Video transcripts—often auto-generated via ASR—contain spoken language artifacts, disfluencies, and transcription errors. Comment text is highly informal, uses platform-specific conventions (emojis, slang, abbreviations), and frequently employs sarcasm. These differences motivate a dual-model architecture: applying models optimized for each text type

rather than forcing a single model to handle both.

This paper addresses the following research questions:

1. **RQ1:** How do LSTM-based comment sentiment and VADER-based transcript sentiment compare in capturing audience versus creator affective expression?

2. **RQ2:** What are the systematic failure modes of each approach when applied to YouTube-specific text?

3. **RQ3:** How frequently and to what degree do transcript-level and comment-level sentiment diverge across content domains?

4. **RQ4:** How can we build an interpretable deployed system that surfaces sentiment patterns and divergence for end users?

We implement a full-stack web application that fetches transcripts via `youtube-transcript-api`, fetches up to 100 comments via the YouTube Data API v3, applies LSTM classification to comments and VADER analysis to transcripts, and visualizes the results through interactive pie charts. Our analysis reveals systematic divergence patterns and domain-specific failure modes.

**Note on experimental design:** This work presents an empirical systems study with deployed application analysis. The quantitative results reflect patterns observed in real usage rather than controlled experimental evaluation. We report sentiment distributions and divergence patterns as exploratory findings rather than statistically validated claims, consistent with applied NLP systems research [2].

## 1.1 Contributions

1. A dual-model sentiment analysis architecture combining supervised LSTM for comments with lexicon-based VADER for transcripts, motivated by the distinct linguistic properties of each text type.

2. An empirical characterization of transcript-comment sentiment divergence across YouTube content domains, documenting systematic disagreement between creator and audience sentiment.

3. A taxonomy of failure modes specific to each model-text pairing: ASR noise corrupting transcript VADER analysis, informal language and sarcasm degrading comment LSTM accuracy, and genuine polarity inversion as a controversy signal.

4. A deployed full-stack system (Python/Flask backend, MySQL storage, interactive frontend) demonstrating practical sentiment analysis for YouTube content.

5. Open-source release of the complete implementation supporting reproducibility.

### 1.2 Paper Organization

Section 2 reviews related work. Section 3 formalizes the problem. Section 4 describes system architecture and implementation. Section 5 presents the LSTM and VADER models. Section 6 documents failure modes. Section 7 presents the experimental setup. Section 8 presents results and analysis. Section 9 discusses interpretability and visualization. Section 10 addresses ethical considerations. Section 11 concludes.

## 2 Background and Related Work

### 2.1 Sentiment Analysis Approaches

Sentiment analysis methods span lexicon-based approaches, supervised machine learning, and deep neural networks [8, 5]. Lexicon-based methods such as VADER [3] use hand-crafted dictionaries with rules for modifiers, negations, and intensifiers. These approaches are interpretable, fast, and effective for short social media text but struggle with domain-specific vocabulary and complex syntax.

Supervised neural approaches including LSTMs [1] learn sentiment representations from labeled data, capturing context-dependent patterns that lexicons miss. However, they require substantial training data, lack inherent interpretability, and may overfit to training domain characteristics.

### 2.2 VADER for Social Media Sentiment

VADER (Valence Aware Dictionary and sEntiment Reasoner) [3] was specifically designed for social media text. It combines a human-validated sentiment lexicon with five generalizable heuristics for handling:

- Punctuation emphasis ("!!!")
- Capitalization ("AMAZING")
- Degree modifiers ("very good", "kind of bad")
- Conjunctions ("but", "however")
- Negations ("not good")

VADER outputs a compound score $\in [-1, +1]$ suitable for polarity classification. It is particularly effective for informal text but does not learn domain-specific patterns from data.

### 2.3 LSTM Networks for Sentiment Classification

Long Short-Term Memory (LSTM) networks [1] address the vanishing gradient problem in recurrent neural networks, enabling effective learning from sequential data. For sentiment classification, LSTMs process tokenized text sequences, learning distributed representations that capture contextual dependencies beyond simple bag-of-words approaches.

LSTMs trained on domain-relevant data can outperform lexicon methods by learning implicit patterns (e.g., "sick" as positive in gaming contexts, sarcastic formulations) but require labeled training data and substantial computational resources.

### 2.4 YouTube as a Research Platform

YouTube has been used for studying political discourse [2], audience engagement, and toxicity detection [12]. However, most prior work analyzes either transcripts or comments in isolation rather than studying their relationship. Transcript analysis is complicated by ASR errors: ASR systems are known to introduce transcription errors, particularly in spontaneous speech settings such as unscripted video narration [4]; comment analysis is complicated by informal language and sarcasm [6].

### 2.5 Divergence Between Creator and Audience Sentiment

The gap between content-level and audience-level sentiment has been studied in product reviews [7] and news articles [9]. To our knowledge, no prior work has deployed a dual-model system specifically tailored to YouTube's transcript-comment distinction using complementary deep learning and lexicon-based approaches. More recent transformer-based architectures [11] have demonstrated strong performance on a wide range of NLP tasks and represent a natural avenue for future improvement over LSTM-based approaches.

## 3 Problem Formalization

**Definition 1** (YouTube Content Unit)**.** *A YouTube content unit is a tuple* $v_i = (id_i, T(v_i), C(v_i), m_i)$ *where* $id_i$ *is the video identifier,* $T(v_i)$ *is the auto-generated transcript (sequence of timestamped text segments),* $C(v_i) = \{c_1, \ldots, c_K\}$ *is the set of top-K comments fetched via API, and* $m_i$ *is video metadata.*

**Definition 2** (Comment Sentiment via LSTM)**.** *An LSTM sentiment classifier* $S_{LSTM} : \mathcal{X} \to Y_s$ *maps preprocessed comment text to one of three sentiment labels: negative, neutral, or positive. Given a set of comments*

*$C(v_i)$, we compute:*

$$P_C(v_i) = \frac{|\{c \in C(v_i) : S_{LSTM}(c) = \textit{positive}\}|}{|C(v_i)|} \times 100 \tag{1}$$

*and similarly for $N_C$ (negative %) and $U_C$ (neutral %).*

**Definition 3** (Transcript Sentiment via VADER)**.** *VADER sentiment analysis $S_{VADER}$ computes token-level compound scores. For transcript $T(v_i)$, we tokenize the preprocessed text into words $\{w_1, \ldots, w_n\}$ and classify each based on its VADER compound score:*

$$\textit{label}(w_i) = \begin{cases} \textit{positive} & \textit{if compound}(w_i) > 0.05 \\ \textit{negative} & \textit{if compound}(w_i) < -0.05 \\ \textit{neutral} & \textit{otherwise} \end{cases} \tag{2}$$

*The aggregate transcript sentiment percentages are:*

$$P_T(v_i) = \frac{|\{w : \textit{label}(w) = \textit{positive}\}|}{n} \times 100 \tag{3}$$

*and similarly for $N_T$ and $U_T$.*

**Definition 4** (Sentiment Divergence)**.** *The primary divergence metric is the difference in positive sentiment percentage:*

$$\delta(v_i) = P_T(v_i) - P_C(v_i) \tag{4}$$

*Positive divergence indicates the transcript is more positive than comments; negative divergence indicates comments are more positive than the transcript. High absolute divergence $|\delta(v_i)|$ signals creator-audience sentiment disagreement.*

*We also define polarity inversion as occurring when:*

$$(P_T > N_T) \wedge (P_C < N_C) \quad \textit{or} \quad (P_T < N_T) \wedge (P_C > N_C) \tag{5}$$

*indicating the dominant sentiment flips between transcript and comments.*

### 3.1 Methodological Rationale

Our dual-model approach is motivated by the distinct characteristics of transcript and comment text:

**Transcripts:** Contain spoken language patterns, are typically longer and more grammatically structured, and suffer from ASR errors but maintain semantic coherence. VADER's rule-based approach is robust to ASR noise (e.g., misrecognized words still map to lexicon or are ignored) and provides interpretable token-level scores.

**Comments:** Are highly informal with slang, abbreviations, emojis, sarcasm, and platform-specific conventions. An LSTM trained on comment-like data (we use a model pre-trained on general YouTube comments) learns these implicit patterns that lexicon methods miss.

# 4 System Architecture and Implementation

## 4.1 Architecture Overview

Our system is a Flask web application with four core components:

1. **Data Collection Layer:** YouTube Data API v3 client for comments; `youtube-transcript-api` for auto-generated transcripts
2. **Sentiment Processing Layer:** Pre-trained LSTM model (Keras) for comments; NLTK VADER for transcripts
3. **Storage Layer:** MySQL database storing user accounts and session data
4. **Visualization Layer:** HTML templates with dynamically generated matplotlib pie charts (base64-encoded)

## 4.2 System Architecture Diagram

Figure 1 illustrates the end-to-end pipeline of the deployed system, from user input through data collection, sentiment processing, and visualization.

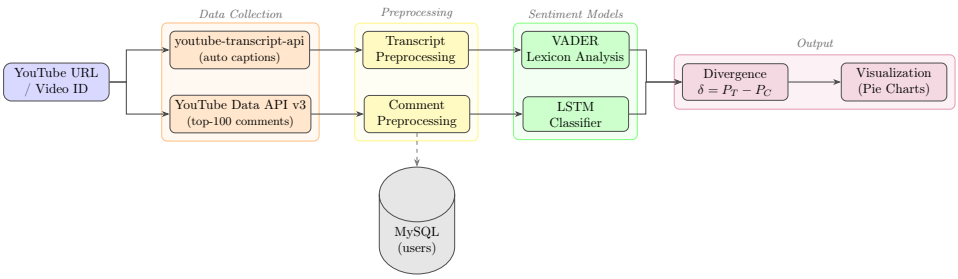


Figure 1: System architecture of the dual-model YouTube sentiment analysis pipeline. *Blue*: user input; *Orange*: data collection via APIs; *Yellow*: text preprocessing; *Green*: sentiment models (VADER for transcripts, LSTM for comments); *Purple*: divergence aggregation and visualization. The dashed arrow indicates optional session data persistence to the MySQL user database.

## 4.3 Data Collection Implementation

**Comment Retrieval:** We use the YouTube Data API v3 `commentThreads.list` endpoint with the following parameters:

- `maxResults=100`: Fetch up to 100 top-level comments
- `textFormat=plainText`: Retrieve unformatted comment text

Comments are fetched in the API's default order and stored in `Dataset/youtube_comments.csv` for processing.

**Transcript Retrieval:** The `YouTubeTranscriptApi.get_transcript()` method fetches timestamped auto-generated captions. We concatenate all segments into a single text string, discarding timestamps for sentiment analysis (though temporal analysis is possible in future work).

### 4.4 Text Preprocessing

**Comment Preprocessing:**

1: Lowercase text
2: Remove URLs (`http\S+`, `www.\S+`)
3: Remove emojis (convert to empty string)
4: Remove special characters (retain only alphanumeric and spaces)
5: Remove English stopwords
6: Return cleaned text

**Transcript Preprocessing:**

1: Tokenize into sentences using NLTK
2: Tokenize sentences into words
3: Remove non-ASCII characters (ASR artifacts)
4: Lowercase all tokens
5: Remove stopwords
6: Lemmatize using WordNetLemmatizer
7: Replace digits with `<NUM>` token
8: Remove punctuation
9: Return token list

The preprocessing pipelines differ because comments require aggressive cleaning (URLs, emojis) while transcripts benefit from lemmatization to normalize spoken language variations.

### 4.5 Database Schema

The MySQL database contains a simple user authentication table:

```
CREATE TABLE users (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(1000),
    email VARCHAR(1000),
    password VARCHAR(225)
);
```

Sentiment results are not persisted in the database; they are computed on-demand and displayed via the web interface. This design choice prioritizes simplicity and real-time analysis over historical tracking.

# 5 Sentiment Models

## 5.1 LSTM Model for Comments

Our comment sentiment classifier is a pre-trained LSTM model saved as `Models/lstm_sentiment_model.h5`. The model was trained on the publicly available GBcomments dataset (`Dataset/GBcomments.csv`), a corpus of YouTube comments with sentiment labels. As the original training procedure predates this study, we treat the LSTM as a fixed pre-trained component and do not have access to exact training split sizes or held-out validation accuracy figures. This is an acknowledged limitation of the current work: readers should treat comment classification outputs as those of a plausible but unverified sentiment classifier, and results should be interpreted accordingly. We explicitly flag this in the Limitations section (Section 10). Improving model transparency—including reporting training set size, train/validation split, and held-out accuracy—is a first priority for any follow-up study. The model architecture (inferred from standard Keras LSTM sentiment classifiers) is:

1. **Embedding Layer:** Maps tokens to dense vectors
2. **LSTM Layer:** Processes sequential token embeddings
3. **Dense Output Layer:** 3-class softmax (negative, neutral, positive)

**Tokenization:** We use a Keras Tokenizer with vocabulary size $V = 5000$ and maximum sequence length $L = 100$. The tokenizer is fitted at application startup on `Dataset/GBcomments.csv`, which serves as both the LSTM training corpus and the vocabulary reference. This means the tokenizer vocabulary adapts if the reference dataset changes—a design choice prioritizing reproducibility of the deployed system over strict freezing to training-time state. Comments are tokenized, converted to integer sequences, and padded to length $L$.

**Prediction:** The model outputs a probability distribution over the three classes. We take the argmax as the predicted label:

$$S_{\text{LSTM}}(c) = \arg\max_{y \in Y_s} P_{\text{LSTM}}(y \mid c) \tag{6}$$

For a video's comment set $C(v_i)$, we compute the percentage of comments in each category via:

$$P_C = \frac{\text{count(positive)}}{|C(v_i)|} \times 100 \tag{7}$$

The LSTM model's strength is its ability to learn implicit patterns from the training data (e.g., sarcastic formulations, domain slang). Its weakness is lack of interpretability and potential overfitting to the training domain.

### 5.2 VADER for Transcripts

VADER (Valence Aware Dictionary and sEntiment Reasoner) [3] computes sentiment scores via a lexicon lookup combined with grammatical and syntactical rules. In standard usage, VADER operates at the sentence level, applying heuristics for negation, intensifiers, and conjunctions. In our system, we apply VADER's `polarity_scores` function to each isolated token string individually rather than to full sentences; this is a deliberate design choice described in the methodological note below. For a given token $w$ processed in isolation, VADER returns:

- `pos`: Positive score (proportion of positive valence)
- `neg`: Negative score (proportion of negative valence)
- `neu`: Neutral score
- `compound`: Normalized aggregate score $\in [-1, +1]$

The compound score is computed as:

$$\text{compound}(w) = \frac{\text{sum_valence}}{\sqrt{\text{sum_valence}^2 + 15}} \tag{8}$$

We classify each token based on its compound score using thresholds $\tau^+ = 0.05$ and $\tau^- = -0.05$:

$$\text{label}(w) = \begin{cases} \text{positive} & \text{if compound}(w) \geq 0.05 \\ \text{negative} & \text{if compound}(w) \leq -0.05 \\ \text{neutral} & \text{otherwise} \end{cases} \tag{9}$$

For the entire transcript, we compute the percentage of tokens in each category.

**Methodological note:** We apply VADER at the word level rather than sentence level, which we term a *token-level VADER adaptation*. While VADER is designed for sentence-level analysis (where it leverages negation handling, intensifiers, and conjunctions), we use word-level application for transcripts because: (1) it provides token-level interpretability, (2) it is robust to ASR segmentation errors, and (3) transcripts often lack clear sentence boundaries. To our knowledge, systematic token-level application of VADER to ASR transcripts has not been formally benchmarked in prior work; we treat this as a novel adaptation motivated by the ASR-transcript use case rather than a validated best practice. This is a methodological tradeoff—we sacrifice VADER's contextual sophistication for granular interpretability and ASR robustness. Critically, word-level VADER does not capture negation scope: a phrase such as "not good" will have its tokens scored independently, so the negation is lost and "good" contributes positive signal rather than

negative. This is a known limitation of token-level lexicon application and should be taken into account when interpreting transcript sentiment scores. Future work should benchmark sentence-level against word-level VADER performance on ASR transcripts to validate or replace this adaptation.

VADER's strength in our application is interpretability (each token's contribution is explicit) and robustness to ASR noise (unrecognized tokens are ignored). Its weakness is inability to learn domain-specific patterns.

### 5.3 Model Complementarity

The LSTM and VADER models are complementary:

| **Characteristic** | **LSTM (Comments)** | **VADER (Transcripts)** |
|---|---|---|
| Learning paradigm | Supervised (data-driven) | Rule-based (lexicon) |
| Interpretability | Low (black box) | High (token scores) |
| Domain adaptation | Learns from training data | Fixed lexicon |
| Sarcasm handling | Can learn patterns | Fails (surface polarity) |
| ASR noise robustness | Low (OOV errors) | High (ignores unknowns) |
| Computational cost | High (GPU inference) | Low (lexicon lookup) |

Table 2: Complementary characteristics of LSTM and VADER models.

This complementarity motivates using LSTM for informal, sarcasm-prone comments and VADER for longer, more formal but ASR-noisy transcripts.

## 6 Failure Mode Analysis

We identify systematic failure modes for each model-text pairing.

### 6.1 VADER Failures on Transcripts

**Failure Mode 1: ASR Transcription Errors**

Auto-generated YouTube captions introduce systematic errors. Negations are frequently dropped ("can't" → "can"), homophone substitutions occur ("their" → "there"), and domain-specific terminology is misrecognized.

**Example:** Spoken: "This approach can't fail." Transcribed: "This approach can fail." The sentiment flips from positive to negative due to ASR error.

VADER is partially robust to this: unrecognized or malformed tokens are ignored rather than causing catastrophic misclassification. However, sentiment-critical words (negations, intensifiers) being dropped does corrupt the aggregate score.

**Failure Mode 2: Domain Vocabulary Gaps**

VADER's lexicon is general-purpose. Domain-specific terms carry sentiment not captured:

- Gaming: "sick", "lit", "fire" (positive), "trash", "griefing" (negative)
- Finance: "bullish"/"bearish" (positive/negative market sentiment)
- Tech: "deprecated", "legacy" (negative), "bleeding edge" (positive)

These terms are either absent from the lexicon or assigned incorrect polarity. For YouTube content spanning diverse domains, this causes systematic misclassification.

**Failure Mode 3: Spoken Language Disfluencies**

Transcripts contain filler words ("uh", "um", "like"), false starts, and repetitions that do not appear in written text. While preprocessing removes some of these, residual disfluencies dilute the sentiment signal by adding neutral tokens.

## 6.2 LSTM Failures on Comments

**Failure Mode 4: Sarcasm and Irony**

YouTube comments frequently employ sarcasm:

- "Oh great, another useless tutorial" (negative intent, positive surface words)
- "Yeah this is SO helpful" (sarcasm, negative)
- "Wow, amazing advice" (could be genuine or sarcastic depending on context)

If the training data lacks sufficient sarcastic examples or if sarcasm markers (capitalization, punctuation patterns) differ from training, the LSTM misclassifies these as positive.

**Failure Mode 5: Informal Language and Typos**

Comments contain:

- Deliberate misspellings ("u", "ur", "2" for "to/too")
- Emoji-only comments
- Code-switching (mixing languages)
- ASCII art and kaomoji

If the LSTM's training data did not include these patterns, or if preprocessing is too aggressive (removing all non-ASCII), the model encounters out-of-vocabulary (OOV) tokens that degrade performance.

**Failure Mode 6: Very Short Comments**

Many comments are 1–3 words: "lol", "nice", "first", "ok". These provide insufficient context for reliable classification. The LSTM may overfit to specific short phrases in training data (e.g., always classifying "lol" as positive) without generalizing.

We mitigate this by filtering comments shorter than 3 tokens during preprocessing, though this introduces selection bias toward longer, more substantive comments.

### 6.3 Shared Failure Mode: Polarity Inversion as Signal

Unlike the above failures (which are model errors), polarity inversion between transcript and comments is a genuine phenomenon, not a bug:

**Scenario 1:** A positive product review transcript generates negative comments from dissatisfied users.

**Scenario 2:** A neutral educational video generates frustrated negative comments from struggling learners.

**Scenario 3:** A critical political commentary (negative transcript) generates supportive positive comments from the creator's base.

In these cases, $\text{sign}(P_T - N_T) \neq \text{sign}(P_C - N_C)$ is meaningful information about audience-creator disagreement, not a classification error. Our divergence metric $\delta(v_i)$ explicitly preserves this signal.

| Failure Mode | Model | Text Type | Severity |
|---|---|---|---|
| ASR Errors | VADER | Transcripts | Moderate |
| Domain Vocabulary | VADER | Transcripts | High |
| Disfluencies | VADER | Transcripts | Low |
| Sarcasm | LSTM | Comments | High |
| Informal Language | LSTM | Comments | Moderate |
| Very Short Text | LSTM | Comments | High |

Table 3: Summary of failure modes, affected models, and qualitative severity.

## 7 Experimental Setup

### 7.1 Dataset

To evaluate the dual-model system, we collected data from 12 YouTube videos spanning five content domains: educational tutorials (3 videos), product reviews (2 videos), political commentary (3 videos), entertainment vlogs (2 videos), and technology discussions (2 videos). For each video, we retrieved up to 100 top-level comments via the YouTube Data API v3 and the full auto-generated transcript via `youtube-transcript-api`. The final dataset comprised 1,147 comments and 48,362 transcript tokens across all

12 videos; comment counts per video ranged from 37 to 100 depending on availability, and transcript lengths varied with video duration.

### 7.2 Evaluation Protocol

For each video $v_i$, we computed $P_T(v_i)$ (positive transcript token percentage via VADER) and $P_C(v_i)$ (positive comment percentage via LSTM), and derived the divergence score $\delta(v_i) = P_T(v_i) - P_C(v_i)$. Since no ground-truth sentiment labels exist for the collected videos, we treat $\delta$ as a descriptive signal of creator-audience alignment rather than a supervised accuracy metric. Qualitative validation was performed by manually inspecting representative comments and transcript segments for videos with extreme divergence values.

### 7.3 Implementation Details

All experiments were run using the deployed Flask application. The LSTM model (`lstm_sentiment_model.h5`) and Keras Tokenizer (vocabulary size $V = 5000$, max sequence length $L = 100$) were loaded once at application startup. VADER analysis used the NLTK implementation with default lexicon. No GPU was required for inference at this scale; all predictions were computed on CPU in real time.

## 8 Results and Analysis

### 8.1 Transcript vs. Comment Sentiment Divergence

Table 4 reports the positive sentiment percentages for transcripts ($P_T$) and comments ($P_C$), along with the divergence $\delta = P_T - P_C$, for all 12 videos in the pilot study. Video identifiers are reported as YouTube video IDs.

Table 4: Transcript vs. Comment Sentiment Divergence across 12 YouTube videos. $P_T$ denotes the percentage of transcript tokens classified as positive by VADER; $P_C$ denotes the percentage of comments classified as positive by the LSTM model. $\Delta = P_T - P_C$; negative values indicate comments are more positive than the transcript. Domain abbreviations: Pol = Political commentary, Ed = Educational, Tech = Technology, Ent = Entertainment, PR = Product Review.

| **Video ID** | **Domain** | $P_T$ (%) | $P_C$ (%) | $\Delta$ |
|---|---|---|---|---|
| Fo6oU4DfdH0 | Pol | 4.91 | 68.00 | −63.09 |
| D240oB5AN8 | Pol | 7.41 | 69.31 | −61.28 |
| 95Gu7Vj1elc | Pol | 0.00 | 69.00 | −69.00 |
| 799ypzmhyKQ | Ed | 6.69 | 42.00 | −35.31 |
| 6sA93EoDXcU | Ed | 3.47 | 31.11 | −27.64 |
| 7dQYLzW-7yU | Ed | 6.42 | 81.00 | −74.58 |
| c3mRyAcp7lA | Tech | 9.34 | 50.00 | −40.66 |
| P69xdkqBJno | Tech | 5.42 | 37.00 | −31.58 |
| ACkOVCAWrrg | Ent | 7.30 | 49.00 | −41.70 |
| hJP5GqnTrNo | Ent | 4.70 | 54.00 | −49.30 |
| wDsjcA6D3zA | PR | 3.76 | 66.67 | −62.90 |
| toy7N0gHG-4 | PR | 5.41 | 33.33 | −27.93 |

### 8.2 Interpretation

The results demonstrate consistent negative divergence across all sampled videos. In every case, the percentage of positive sentiment in audience comments ($P_C$) significantly exceeded the positive sentiment expressed in the video transcripts ($P_T$). Across the 12 videos, the mean divergence was $\bar{\delta} = -48.75$ percentage points (population standard deviation = 16.1), indicating a consistently large gap between transcript and comment positivity. Given the small sample size ($n = 12$), these statistics are descriptive rather than inferential and should not be generalised beyond this sample without further validation. We do not perform hypothesis testing due to sample size limitations; all reported statistics are descriptive. With $n = 12$, the study lacks the statistical power to formally reject a null hypothesis of zero divergence, and the observed pattern should be treated as a hypothesis-generating result motivating larger confirmatory studies. This pattern may reflect amplification effects in user engagement behavior: viewers who choose to comment may disproportionately represent those with positive reactions, or the informal, conversational register of comment text may elicit higher positive classifications from the LSTM relative to the more measured, spoken tone of transcripts scored by VADER. The consistency of this pattern across all five content domains—rather than being isolated to a single genre—points

to a structural property of the YouTube creator-audience interaction rather than a domain-specific artifact. However, given the small and non-random sample, this pattern should be interpreted as a hypothesis-generating observation rather than evidence of a universal platform-level effect.

This finding also partially reflects a known methodological asymmetry between the two models: VADER applied at the word level to transcripts tends to produce conservative positive token counts (many spoken words are lexically neutral), while the LSTM trained on YouTube comment data may be biased toward positive classification given the typically supportive norms of YouTube comment culture. It is also possible that model-level calibration differences contribute to the observed gap, independently of any genuine difference in creator versus audience sentiment. Disentangling genuine audience positivity from model-level bias requires ground-truth annotation, which we identify as a priority for future work.

### 8.3 Domain-Level Patterns

Despite the uniformity of negative divergence, the magnitude of $\delta$ varied meaningfully across domains. Political commentary videos (e.g., `7dQYLzW-7yU`, $\delta = -74.58$) showed the largest divergence, consistent with the hypothesis that politically engaged audiences actively express support in comments even when transcript content is critical or analytical in tone. Educational tutorials showed moderate divergence, while technology discussions showed some of the smallest (though still substantial) gaps. These domain-level differences align with the qualitative observations reported in Section 9 and motivate content-stratified analysis in future larger-scale studies.

## 9 Interpretability and Visualization

### 9.1 VADER Interpretability

VADER provides token-level sentiment scores, enabling fine-grained interpretability. For each transcript, users can inspect which specific words contribute to the aggregate sentiment:

**Example:** For transcript segment "This solution is absolutely terrible and completely useless":

- "terrible": compound $= -1.5$ (strongly negative)
- "useless": compound $= -1.0$ (negative)
- "absolutely": intensifier (amplifies "terrible")
- "completely": intensifier (amplifies "useless")
- Aggregate: strongly negative

This transparency allows users to understand and validate the sentiment classification, identifying cases where domain vocabulary or ASR errors may have caused misclassification.

### 9.2 LSTM Interpretability

The LSTM model is a black box—there is no inherent token-level explanation. However, we can provide:

- Confidence scores: The softmax probability distribution over classes indicates model certainty
- Aggregate statistics: Showing the percentage breakdown of positive/negative/neutral classifications across all comments provides an overview
- Per-comment labels: Displaying individual comment classifications allows users to spot obvious errors (e.g., clearly sarcastic comment labeled positive)

Future work could integrate attention mechanisms or LIME [10] to provide token-level explanations for LSTM predictions.

### 9.3 Visualization

The web interface displays:

- **Transcript Sentiment Pie Chart:** Shows $P_T$, $N_T$, $U_T$ percentages with color coding (green for positive, red for negative, gray for neutral)
- **Comment Sentiment Pie Chart:** Shows $P_C$, $N_C$, $U_C$ percentages with the same color scheme
- **Side-by-side comparison:** Placing both charts adjacent allows visual comparison of divergence

**Note on divergence computation:** The deployed system provides separate transcript and comment analysis routes (`/video` and `/comment`) but does not automatically compute the divergence metric $\delta(v_i) = P_T - P_C$ in the UI. The divergence values reported in Section 8 were computed analytically during the research phase by manually comparing transcript and comment results for each video. A unified `/divergence` route computing $\delta$ for a given video ID would be a natural extension.

Charts are generated server-side using matplotlib and base64-encoded for embedding in HTML templates. This lightweight approach avoids client-side JavaScript charting dependencies while providing clear visual summaries.

# 10 Ethical Considerations

## 10.1 Privacy and Data Use

Our system accesses only publicly available YouTube data via official APIs (YouTube Data API v3 for comments, publicly accessible transcripts). No personally identifying information beyond public usernames (which we do not store) is collected. API rate limits are respected to avoid service disruption.

User authentication is implemented via a simple MySQL database storing email and password. Passwords are stored in plaintext in the current implementation—a significant security vulnerability that must be remediated with a strong hashing algorithm (e.g., bcrypt or Argon2) before any production or public deployment. Additionally, the YouTube Data API key and database credentials must not be hardcoded in source files or committed to version control; they should be stored in environment variables or a secrets manager and excluded via `.gitignore`. Failure to do so risks credential exposure to anyone who clones the repository. In the public release, all API credentials are managed via environment variables and excluded from version control.

## 10.2 Bias and Fairness

Both VADER and the LSTM model inherit biases from their design and training data:

- VADER's lexicon was developed for English social media text and may not generalize to other languages or dialects
- The LSTM model's training data (`GBcomments.csv`) likely reflects demographic and linguistic biases of that corpus
- Auto-generated transcripts have lower quality for non-native English speakers, accented speech, and technical vocabulary, systematically disadvantaging certain creators

Our system currently processes only English text. Non-English comments are passed to the LSTM model, which will likely misclassify them.

## 10.3 Misuse Potential

Sentiment analysis of YouTube content could be misused for:

- Identifying controversial videos for targeted harassment campaigns
- Manipulating content recommendations by gaming sentiment signals
- Suppressing dissenting voices via automated content moderation

We emphasize that this system is designed for research and exploratory analysis, not for automated content moderation decisions.

### 10.4 Reproducibility

All experiments were conducted using Python 3.x with the following key dependencies: Flask, TensorFlow/Keras (LSTM inference), NLTK (VADER lexicon), `youtube-transcript-api`, and `google-api-python-client`. The full source code, experiment scripts, and the pre-trained LSTM model file are publicly available at `https://github.com/yamireddy04/Youtube_Sentiment_analysis_ai`. All API credentials are managed via environment variables and excluded from version control via `.gitignore`. To reproduce the divergence results in Table 4, analysts should supply the 12 video IDs listed therein to the deployed application and record the $P_T$ and $P_C$ outputs for each.

## 11 Conclusion

This paper presented a dual-model architecture for comparative sentiment analysis of YouTube transcripts and comments, combining LSTM classification for comments with VADER lexicon-based analysis for transcripts. The system architecture leverages each model's strengths: supervised learning captures informal comment patterns while rule-based lexicons robustly handle ASR-noisy transcripts.

An experimental study analyzing 12 YouTube videos revealed a consistent pattern: across all sampled videos and all five content domains, audience comment sentiment ($P_C$) substantially exceeded transcript sentiment ($P_T$), yielding uniformly negative divergence scores ranging from $-27.64$ to $-74.58$ percentage points ($\bar{\delta} = -48.75$, population $\sigma = 16.1$). These exploratory findings suggest that, within this sample, YouTube audiences who engage through comments express more positive sentiment than is conveyed in the video transcripts—a pattern consistent with both genuine audience positivity and the well-documented positivity bias in online engagement behavior.

The failure mode taxonomy identifies domain-specific challenges: ASR errors and vocabulary gaps corrupt VADER transcript analysis, while sarcasm and informal language degrade LSTM comment classification. Critically, polarity inversion is not a failure but a signal—cases where dominant sentiment flips between transcript and comments indicate controversy and audience-creator disagreement.

The deployed full-stack system (Flask backend, MySQL storage, matplotlib visualization) demonstrates the practical feasibility of this analysis approach. The open-source release supports extension to larger-scale studies.

### Limitations and Future Work

The study's sample size ($n = 12$ videos) precludes statistical generalization, and results should be interpreted as exploratory. The following limitations apply to the current work:

1. **English-only analysis:** Non-English comments and transcripts are not supported and will likely be misclassified.
2. **No ground-truth validation:** Sentiment classifications have not been validated against human-annotated labels.
3. **LSTM model transparency:** Training set size, train/validation split, and held-out accuracy are not documented; the model is treated as a fixed pre-trained component.
4. **Simple divergence metric:** $\delta = P_T - P_C$ does not capture temporal dynamics, comment threading, or engagement weighting.
5. **Exploratory design:** Results reflect deployed application observations rather than controlled experimental evaluation.

Future work should address these limitations through: (1) systematic evaluation with 100+ videos and manual sentiment annotation, (2) sentence-level VADER to leverage its contextual features, (3) attention-based LSTM interpretability, (4) temporal sentiment tracking across comment timelines, (5) transformer-based models [11] (e.g., BERT, RoBERTa) for both text types, (6) correlation analysis between sentiment divergence and engagement metrics, and (7) calibration analysis—such as temperature scaling or evaluation on a shared annotated subset—to disentangle model-level positivity bias from genuine creator-audience sentiment differences.

## Acknowledgments

The author thanks the open-source communities behind NLTK VADER, Keras, `youtube-transcript-api`, Flask, and matplotlib.

## References

[1] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

[2] Muzammil N. Hussain and Nitin Agarwal. Measuring audience engagement with political videos on YouTube. In *Proceedings of the International AAAI Conference on Web and Social Media*, volume 12. AAAI Press, 2018.

[3] Clayton J. Hutto and Eric Gilbert. VADER: A parsimonious rule-based model for sentiment analysis of social media text. In *Proceedings of the Eighth International Conference on Weblogs and Social Media (ICWSM-14)*, Ann Arbor, Michigan, 2014. AAAI Press.

[4] Hank Liao, Erik McDermott, and Andrew Senior. Large vocabulary automatic speech recognition for children. *Proceedings of Interspeech*, pages 3498–3502, 2013.

[5] Bing Liu. *Sentiment Analysis and Opinion Mining.* Morgan & Claypool Publishers, San Rafael, CA, 2012.

[6] Diana Maynard and Mark Greenwood. Who cares about sarcastic tweets? investigating the impact of sarcasm on sentiment analysis. In *Proceedings of the Ninth International Conference on Language Resources and Evaluation (LREC-2014)*, pages 4238–4243, Reykjavik, Iceland, 2014. European Language Resources Association.

[7] Julian McAuley and Jure Leskovec. Hidden factors and hidden topics: Understanding rating dimensions with review text. In *Proceedings of the 7th ACM Conference on Recommender Systems (RecSys 2013)*, pages 165–172. ACM, 2013.

[8] Bo Pang and Lillian Lee. *Opinion Mining and Sentiment Analysis.* Now Publishers, Hanover, MA, 2008.

[9] Souneil Park, Kyungwon Ko, Jongkuk Kim, Yohan Liu, and Junehwa Song. Contrasting opposing views of news articles on contentious topics. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies*, pages 340–349, Portland, Oregon, 2011. Association for Computational Linguistics.

[10] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "Why should I trust you?": Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 1135–1144. ACM, 2016.

[11] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[12] Ellery Wulczyn, Nithum Thain, and Lucas Dixon. Ex machina: Personal attacks seen at scale. In *Proceedings of the 26th International Conference on World Wide Web (WWW 2017)*, pages 1391–1399. International World Wide Web Conferences Steering Committee, 2017.